

FENIXCMS_8

FASE 8 - CIERRE DEFINITIVO DE LA SEGUNDA REALIDAD Y GOBERNANZA DE PERSISTENCIA

OBJETIVO: PostgreSQL debe ser la unica fuente de verdad del runtime de FenixCMS. Ningun dato demo, Map, Set, INITIAL_*, fallback en memoria, configuracion local o mutacion optimista puede actuar como fuente comercial en production.

REPOSITORIO: fenixcmssl-spec/moreno

RAMA: main

DONDE SE EJECUTA: AI Studio / GitHub. NO desplegar al VPS hasta cerrar todos los criterios.

PRIORIDAD: MAXIMA.

0. AUDITORIA ACTUAL Y MOTIVO

El main actual ya dispone de PostgreSQL obligatorio en production, aislamiento multi-tenant, rechazo del tenant spoofing en varias rutas, Production Gate CI, CheckoutSession/PayPal server-side y provisioning transaccional.

Los bloqueadores que quedan se concentran en la consistencia de estado:

- 1) SaaSProvisioningService mantiene IN_MEMORY_CHECKOUT_SESSIONS y PROVISIONED_CAPTURE_IDS.
- 2) SaaSCheckoutService mantiene dependencias de INITIAL_* y valores de respaldo fuera del camino estrictamente persistente.
- 3) SuperAdminService puede devolver usuarios demo y mantiene defaultSettings en memoria.
- 4) StoreContext inicializa con INITIAL_TENANT/PRODUCTS/PLANS/LICENSES/PLUGINS/THEMES/MEDIA y contiene estado local de negocio.
- 5) El provisioning crea un owner con una password temporal aleatoria, pero no existe en el flujo auditado un onboarding/activation completo para que el cliente establezca su propia password.
- 6) El capture debe comprobar explicitamente custom_id/reference_id frente a CheckoutSession, ademas de order, amount y currency.
- 7) El CI comprueba build y tests pero necesita una auditoria de servicios contra la segunda realidad.
- 8) El schema conserva Float en muchos campos monetarios.

Regla final: si PostgreSQL esta vacio, se muestra vacio; si PostgreSQL esta caido, se devuelve error/503; nunca se muestran datos demo como sustituto.

1. FASE 8A - SUPER ADMIN: POSTGRESQL ONLY

Archivos minimo:

lib/services/super-admin.service.ts
prisma/schema.prisma
app/api/admin/settings/route.ts
app/api/admin/users/route.ts
app/api/admin/metrics/route.ts
app/api/admin/plugins/route.ts
app/api/admin/themes/route.ts
app/api/admin/tenants/route.ts
app/api/admin/licenses/route.ts
app/api/admin/payments/route.ts
app/api/admin/subscriptions/route.ts
app/api/admin/invoices/route.ts

Implementar:

- DB OK + registros -> registros reales.
- DB OK + cero registros -> [] / estado vacio.
- DB ERROR -> 500/503; nunca INITIAL_* ni usuarios demo.
- Eliminar usuarios demo de runtime.
- Eliminar fallback de metricas.
- Eliminar fallback de plugins/themes/licenses/tenants/payments/subscriptions/invoices.

Resultado obligatorio: el panel Super Admin refleja exactamente PostgreSQL.

2. FASE 8B - PLATFORM SETTINGS PERSISTENTES

El estado actual defaultSettings en memoria debe desaparecer como autoridad.

Crear o adaptar:

```
model PlatformSetting {  
  id String @id @default(uuid())
```

```
key String @unique
value Json
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
}
```

API:

GET /api/admin/settings

PUT /api/admin/settings

Flujo:

AUTH -> RBAC -> Zod -> PostgreSQL upsert/transaction -> AuditLog -> response.

Prohibido: modificar solo un objeto JS y devolver success.

3. FASE 8C - STORECONTEXT SIN DEMO STATE EN PRODUCCION

Archivo: lib/storeContext.tsx

Production NO debe inicializar con:

INITIAL_TENANT

INITIAL_PRODUCTS

INITIAL_PLANS

INITIAL_LICENSES

INITIAL_ORDERS

INITIAL_PLUGINS

INITIAL_THEMES

INITIAL_MARKETPLACE_ITEMS

INITIAL_MEDIA_ITEMS

Usar:

loading / null / empty

y despues hidratar desde APIs reales.

Los seeds/demo solo pueden existir bajo una condicion explicita de development/test.

resetToDemoData() debe quedar bloqueado en production.

4. FASE 8D - TODAS LAS MUTACIONES PASAN POR API + DB

Revisar toda funcion de StoreContext relacionada con:

- tenant
- license
- plan
- product
- order
- plugin
- theme
- media
- settings
- marketplace

Regla:

API confirmada -> actualizar UI.

API falla -> rollback/no success.

Un estado optimista es aceptable solo como estado transitorio.

Nunca puede convertirse en fuente de verdad.

5. FASE 8E - CHECKOUTSESSION SIN MEMORIA EN PRODUCTION

Archivo: lib/services/saas-provisioning.service.ts

IN_MEMORY_CHECKOUT_SESSIONS y PROVISIONED_CAPTURE_IDS pueden existir unicamente para tests aislados.

En NODE_ENV=production:

- no leerlos como fallback;
- no depender de ellos para idempotencia;
- no usarlos para declarar COMPLETED.

Fuente de verdad:

CheckoutSession PostgreSQL + Payment providerTransactionId UNIQUE + constraints de CheckoutSession.

Estados recomendados:

CREATED
PENDING_APPROVAL
APPROVED
CAPTURE_PENDING
PROVISIONING
COMPLETED
FAILED
FAILED_RETRYABLE
EXPIRED
CANCELLED

=====

6. FASE 8F - PAYPAL RECONCILIATION Estricta

=====

Archivos:

lib/services/saas-checkout.service.ts
lib/services/paypal-gateway.service.ts
app/api/billing/capture/route.ts
app/api/webhooks/paypal/route.ts

Antes de provisioning comprobar TODOS:

1. session.paypalOrderId == capture.orderId
2. capture ID real y nuevo
3. capture.status == COMPLETED
4. captured amount == session.amountExpected con Decimal
5. captured currency == session.currencyExpected
6. capture custom_id/reference_id == session.id
7. capture ID/provider transaction no usado antes

Cualquier mismatch:

NO PROVISIONING
NO LICENSE
NO SUBSCRIPTION
NO INVOICE PAID

Registrar failureCode.

El return de PayPal nunca es evidencia de pago por si solo.

=====

7. FASE 8G - IDEMPOTENCIA DURABLE Y CONCURRENCIA

=====

Payment.providerTransactionId debe seguir siendo UNIQUE.

CheckoutSession debe tener unique para:

paypalOrderId
paypalCaptureId
paymentId
tenantId cuando se asigne.

Probar:

capture A + capture B simultaneos -> un solo Payment/Tenant/License/Subscription/Invoice.
browser + webhook + retry -> una sola provision.

No depender de Set/Map para garantizar esto.

=====

8. FASE 8H - ONBOARDING REAL DEL OWNER

=====

El provisioning actual genera una password temporal aleatoria. Eso no es un onboarding completo si el usuario nunca recibe un canal para activarla.

Implementar:

User.status = PENDING para el nuevo owner.
Crear token de activacion aleatorio.
Guardar solo hash(token) en PostgreSQL.
Enviar enlace de activacion por email despues del COMMIT.

Usuario define su propia password.

Usuario pasa a ACTIVE.

Usuario puede crear Session.

No enviar ni guardar password temporal en texto plano.

Modelo recomendado:

UserActivationToken:

id, userId, tokenHash UNIQUE, expiresAt, usedAt, createdAt.

Validacion de token:

hash(token) coincide + no expirado + usedAt NULL.

Consumo del token debe ser atomico.

Si email falla despues de DB COMMIT:

mantener cuenta valida y token pendiente; permitir reenvio.

9. FASE 8I - AUTH POST-ACTIVATION

AuthService.login:

1. User desde PostgreSQL.

2. status ACTIVE.

3. PasswordService verifica hash.

4. Session persistida.

5. Cookie httpOnly/secure production.

PENDING y SUSPENDED deben rechazar login.

No password master.

No inMemoryUserStore como authority.

10. FASE 8J - AUDITORIA SOURCE-OF-TRUTH

Crear scripts/audit-production-source-of-truth.ts.

Debe escanear servicios criticos y fallar si encuentra patrones como:

INITIAL_* usado como fuente de runtime.

FALLBACK_* usado en production.

IN_MEMORY_*.

new Map()/new Set() utilizados como ledger de negocio.

usuarios demo.

tenant demo.

settings hardcoded como fuente final.

fake payment state.

success=true despues de catch de DB.

Excepciones:

tests/

docs/

fixtures/

bloques explicitamente marcados development/test-only.

11. FASE 8K - CI PRODUCTION GATE

Modificar .github/workflows/production-gate.yml.

Orden obligatorio:

npm ci

npx prisma generate

npm run lint

npm test

npm run audit:production

npm run audit:source-of-truth

npm run build

El gate debe fallar si aparece una segunda realidad.

12. FASE 8L - DB VACIA Y DB CAIDA

=====

Test DB valida pero sin datos:

- Super Admin vacio.
- Storefront vacio.
- no users demo.
- no tenants demo.
- no plans inventados.

Test DB no disponible:

- login -> 503/500.
- settings -> 503/500.
- checkout -> 503/500.
- provisioning -> 503/500.
- media -> 503/500.
- products/orders/licenses/subscriptions -> 503/500 cuando la DB sea obligatoria.

Nunca devolver demo data como recovery.

=====

13. FASE 8M - RESTART Y MULTIPROCESS

=====

Crear recursos persistentes y reiniciar Node.

Todos deben seguir:

settings, plans, licenses, subscriptions, payments, users, products, orders.

Ejecutar dos procesos contra la misma DB y probar:

same capture

same webhook

same settings update

same idempotency key

Resultado determinista.

=====

14. FASE 8N - RECOVERY DE PAYPAL

=====

Caso A: PayPal COMPLETED + browser cerrado -> webhook/reconciliacion termina provisioning.

Caso B: Browser y webhook llegan -> una sola provision.

Caso C: Payment capturado + DB transaction failure -> CheckoutSession FAILED_RETRYABLE/PROVISIONING recuperable.

Recovery nunca vuelve a cobrar una Order ya capturada.

=====

15. FASE 8O - DINERO

=====

El schema sigue usando Float en multiples campos monetarios.

Prioridad de migracion:

Payment.amount

Invoice.subtotal/tax/total

Subscription.amount

License.price

Product.price

Order.subtotal/tax/total

Para la ruta financiera usar Decimal/Prisma Decimal.

No usar igualdad insegura de Float.

No romper migraciones historicas.

=====

16. FASE 8P - CRITERIOS DE ACEPTACION

=====

[] SuperAdmin nunca devuelve users demo en production.

[] Settings son persistentes.

[] StoreContext no usa demo state en production.

[] resetToDemoData bloqueado.

[] CheckoutSession no depende de memoria en production.

[] PayPal reference/amount/currency verificadas.

[] browser + webhook = una sola provision.

[] Owner activation funcional.

- [] PENDING no puede login.
- [] DB caída = error; nunca demo.
- [] audit-production-source-of-truth integrado en CI.
- [] multi-process determinista.
- [] Float financiero crítico identificado/migrado.
- [] npm test PASS real.
- [] lint PASS real.
- [] build PASS real.

17. FASE 8Q - TESTS OBLIGATORIOS

Crear tests/fenixcms-fase8.test.ts.

Minimo:

1. DB-empty no demo users.
2. DB-empty no demo settings.
3. DB down -> 500/503.
4. StoreContext production no INITIAL data.
5. resetToDemoData blocked.
6. CheckoutSession durable.
7. duplicate capture -> one Payment.
8. duplicate capture -> one Tenant.
9. duplicate capture -> one License.
10. browser+webhook -> one provisioning.
11. custom_id mismatch -> reject.
12. amount mismatch -> reject.
13. currency mismatch -> reject.
14. activation token one use.
15. expired token reject.
16. PENDING user reject login.
17. ACTIVE user login ok.
18. settings persist.
19. restart preserves state.
20. source-of-truth audit catches new memory ledger.
21. two processes deterministic.

18. PROMPT MAESTRO PARA AI STUDIO

INICIO FASE FENIXCMS_8

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno / rama main

OBJETIVO:

Cerrar definitivamente la segunda realidad del runtime. PostgreSQL es la unica fuente de verdad en production.

REGLAS:

1. Audita primero; no declares PASS por compilar.
2. Elimina usuarios/settings/demo fallbacks del SuperAdminService en production.
3. Persistir PlatformSettings en PostgreSQL.
4. StoreContext: production inicia loading/null/empty; no INITIAL_* como estado real.
5. resetToDemoData bloqueado en production.
6. Todas las mutaciones de negocio usan API + PostgreSQL y solo actualizan UI despues de respuesta confirmada.
7. SaaSProvisioningService no usa IN_MEMORY_* ni Set/Map como autoridad en production.
8. CheckoutSession/PostgreSQL es la autoridad de idempotencia.
9. Capture debe validar orderId, captureId, status, amount, currency y custom_id/reference_id.
10. Browser y webhook deben converger en una provision unica.
11. Owner nuevo entra PENDING_ACTIVATION; crear activation token hash y email de activacion; nunca password temporal perdida.
12. PENDING/SUSPENDED no puede login.
13. Crear audit-production-source-of-truth.ts.
14. Integrarlo en production-gate.yml.
15. Auditar Float financiero y migrar primero Payment/Invoice/Subscription/License a Decimal.
16. Crear tests/fenixcms-fase8.test.ts.
17. Ejecutar realmente npm ci, prisma generate, lint, test, audit:production, audit:source-of-truth y build.
18. No declarar fase terminada si un criterio falla.

INFORME FINAL:

- archivos nuevos/modificados;
- migraciones;
- constraints/indices;
- APIs;
- cambios StoreContext;
- cambios SuperAdmin;
- cambios Checkout/Provisioning;
- onboarding owner;
- tests exactos;
- lint;
- build;
- riesgos restantes;
- confirmacion de que production no usa datos demo como fuente de verdad.

FIN FASE FENIXCMS_8

=====

19. PUERTA DE PRODUCCION DESPUES DE FENIXCMS_8

=====

Esta fase no declara por si sola que todo el producto esta listo.

Despues deben verificarse en el VPS: migraciones PostgreSQL, secretos, PayPal Sandbox E2E, PayPal Live, dominios/SSL, storage real, backups/restore, monitoring, rate limiting distribuido, carga y rollback.

CONCLUSION:

La mejora maxima ahora no es añadir otra feature de CMS. Es convertir FenixCMS en una plataforma donde solo existe una realidad operacional: PostgreSQL + proveedores externos verificados. Mientras haya segunda realidad demo/en memoria en servicios de producción, no debe abrirse la venta a clientes reales.

FIN DEL DOCUMENTO FENIXCMS_8